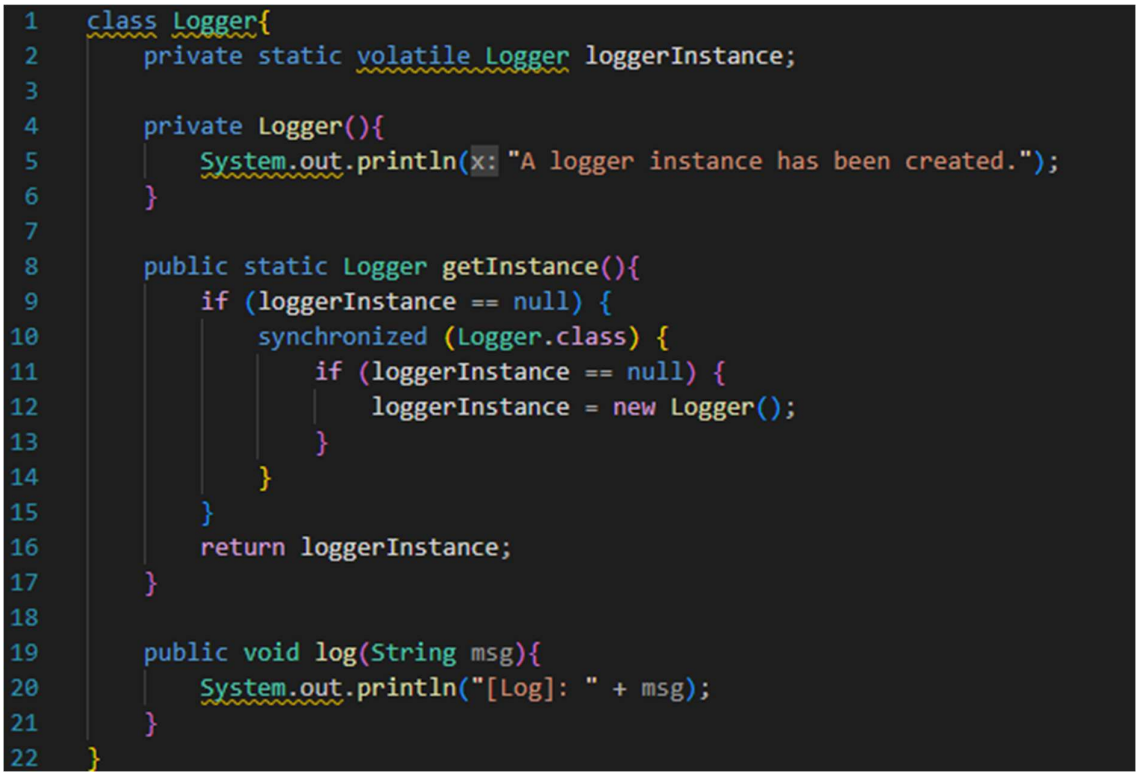
Exercise 1:

Code:

Logger.java:

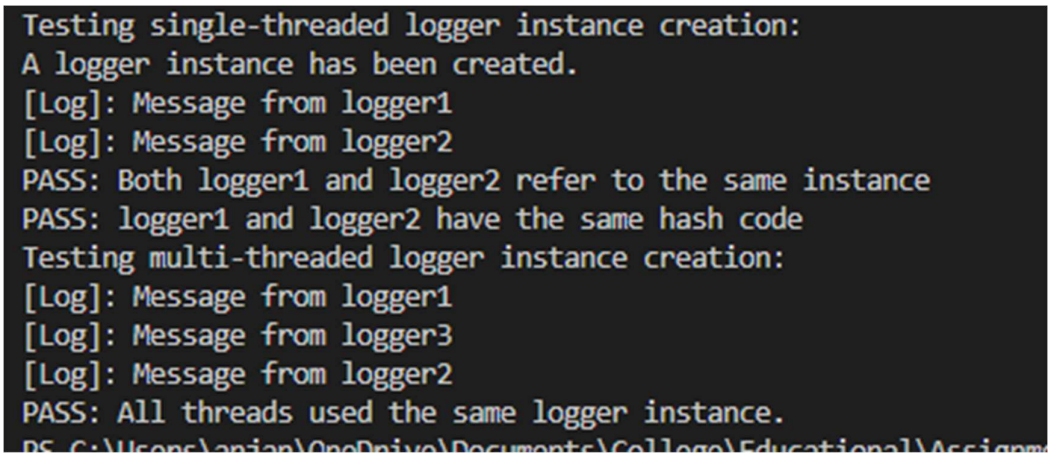


```java
class Logger{
    private static volatile Logger loggerInstance;

    private Logger(){
        System.out.println(x: "A logger instance has been created.");
    }

    public static Logger getInstance(){
        if (loggerInstance == null) {
            synchronized (Logger.class) {
                if (loggerInstance == null) {
                    loggerInstance = new Logger();
                }
            }
        }
        return loggerInstance;
    }

    public void log(String msg){
        System.out.println("[Log]: " + msg);
    }
}
```

LoggerTest.java:

```java
public class LoggerTest {
    public static void main(String[] args) throws Exception {
        testSingleThread();
        testMultiThread();
    }
    private static void testSingleThread() {
        System.out.println(x: "Testing single-threaded logger instance creation:");

        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log(msg: "Message from logger1");
        logger2.log(msg: "Message from logger2");
        if(logger1 == logger2)
            System.out.println(x: "PASS: Both logger1 and logger2 refer to the same instance");
        else
            System.out.println(x: "FAIL: logger1 and logger2 refer to different instances");
        if(logger1.hashCode() == logger2.hashCode())
            System.out.println(x: "PASS: logger1 and logger2 have the same hash code");
        else
            System.out.println(x: "FAIL: logger1 and logger2 have different hash codes");
    }

    private static void testMultiThread() throws InterruptedException{
        System.out.println(x: "Testing multi-threaded logger instance creation:");

        Logger[] loggers = new Logger[3];
        Thread[] threads = new Thread[3];
        for(int i=0; i<3; i++){
            final int idx = i;
            threads[i] = new Thread(() -> {
                loggers[idx] = Logger.getInstance();
                loggers[idx].log("Message from logger" + (idx+1));
            });
        }
        for(Thread t : threads)
            t.start();
        for(Thread t : threads)
            t.join();

        boolean instanceCheck = true;
        for(int i=1; i<3; i++){
            if(loggers[i] != loggers[0]){
                instanceCheck = false;
                break;
            }
        }
        if(instanceCheck)
            System.out.println(x: "PASS: All threads used the same logger instance.");
        else
            System.out.println(x: "FAIL: More than one logger instance was created in different threads.");
    }
}
```

Output:


Testing single-threaded logger instance creation:
A logger instance has been created.
[Log]: Message from logger1
[Log]: Message from logger2
PASS: Both logger1 and logger2 refer to the same instance
PASS: logger1 and logger2 have the same hash code
Testing multi-threaded logger instance creation:
[Log]: Message from logger1
[Log]: Message from logger3
[Log]: Message from logger2
PASS: All threads used the same logger instance.